


5.2 - Algoritmi di instradamento

Ripasso

Comprendiamo quindi che il livello di network è il livello responsabile del trasporto dei segmenti, dei pacchetti da un host (trasmettitore) ad un altro (ricevitore) e per fare ciò vengono svolte due importanti funzioni:

- | | |
|--|--|
| <ul style="list-style-type: none">• Inoltro (Forwarding)
Consiste nel trasferimento dei pacchetti da un router a quello più limitrofo | <ul style="list-style-type: none">• Instradamento (forwarding)
Consiste nel determinare il percorso che i pacchetti devono seguire, la scelta viene fatta tramite i cosiddetti algoritmi di instradamento (algoritmi di routing) |
|--|--|

Gli **algoritmi di instradamento** (routing algorithm) sono degli algoritmi il cui scopo è quello di determinare i percorsi, tra le sorgenti e i destinatari, attraverso la rete di router, percorsi di cui, tipicamente, il migliore è quello che ha “costo minimo”

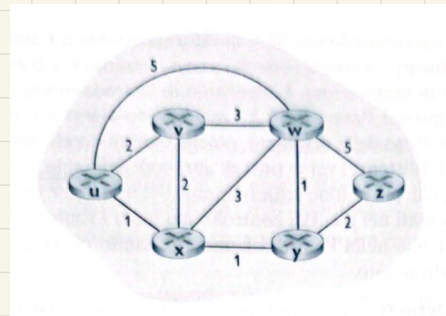
Per formulare i problemi di instradamento si utilizza un **grafo** indicato con la lettera

$G = (N, E)$ che sta ad indicare:

- | | |
|----------------------------------|---|
| • un insieme di N nodi e |] ove ciascun arco collega una coppia di nodi di N |
| • un insieme di E archi (edge) | |

Nel contesto dell'instradamento a livello di rete:

- i nodi del grafo rappresentano i router (che prendono decisioni sull'inoltro dei pacchetti), mentre
- gli archi rappresentano i collegamenti fisici tra i router a cui è associato un valore che ne indica il costo



Costo che in genere può riflettere:

- la lunghezza fisica del collegamento corrispondente o
- la velocità del collegamento o
- il suo prezzo

Per ogni arco (x, y) tra i nodi x e y denotiamo con $c(x, y)$ il suo costo, costo che:

- se la coppia (x, y) appartiene ad E , $c(x, y) =$ un valore
- se la coppia (x, y) NON appartiene ad E , $c(x, y) = \infty$ e
- $c(x, y) = c(y, x)$ perché consideriamo grafi non orientati, cioè con archi bidirezionali ($\text{arco}(x, y) = \text{arco}(y, x)$)

Concludiamo questa infarinatura di concetti aggiungendo che un nodo y viene detto **adiacente** o **vicino** (neighbor) a un nodo x se (x, y) è un arco in E

Ricordando che lo scopo degli algoritmi di instradamento è l'individuazione del percorso migliore, il grafo è un perfetto sistema per aiutare in ciò

Definendo come **percorso** in un grafo $G = (N, E)$ una sequenza di nodi tali che ciascuna delle coppie sia un arco appartenente ad E , il costo di un percorso è la somma di tutti i costi degli archi lungo il percorso

Dati due nodi x e y , esistono più percorsi che li congiungono, ciascuno con il proprio costo, tuttavia, di questi uno o più rappresenta un **percorso a costo minimo** (least-cost path)

Se però tutti gli archi del grafo hanno lo stesso costo, il percorso a costo minimo rappresenterà il **percorso più breve** (shortest path), nonché il percorso con il minor numero di collegamenti tra sorgente e destinazione

Seppur gli algoritmi di instradamento condividano lo stesso scopo, questi possono essere classificati in vario modo a seconda del criterio preso in considerazione :

- Se il criterio è **su quale base calcolano i percorsi a costo minimo**, allora abbiamo:
 - **Algoritmi centralizzati**
Calcolano i percorsi a costo minimo tra una sorgente e una destinazione avendo una conoscenza globale e completa della rete, che si traduce nell'avere informazioni complete su connettività (quali router hanno vicino) e costi (qual è il costo per collegarsi ai router adiacenti), tanto da farli valere il nome di **algoritmi link-state (LS)**

Conoscenza che viene acquisita prima di effettuare il vero e proprio calcolo che può essere svolto:

- o in una sola locazione, quale potrebbe essere un controller logicamente centralizzato
- o replicato in ogni router della rete

- **Algoritmi decentralizzati**
Calcolano i percorsi a costo minimo tra una sorgente e una destinazione in modo distribuito e iterativo

Nessun nodo possiede informazioni complete sul costo di tutti i collegamenti della rete:

- inizialmente i nodi conoscono soltanto i costi dei collegamenti a loro incidenti
- poi, attraverso un processo iterativo e lo scambio di informazioni con i nodi adiacenti, un nodo gradualmente calcola il percorso a costo minimo verso una destinazione o un insieme di destinazioni

(Ricorda questo nome, **algoritmo distance-vector (DV)**)

- Se il criterio è la **frequenza con cui calcolano i percorsi a costo minimo**, allora abbiamo:
 - **Algoritmi di instradamento statici**
Cambiano i percorsi a costo minimo molto raramente, spesso come risultato di un intervento umano
 - **Algoritmi di instradamento dinamici**
Cambiano i percorsi a costo minimo al variare del volume di traffico o della tipologia della rete

Un algoritmo dinamico può essere seguito:
 - sia periodicamente
 - o come conseguenza diretta di un cambiamento nella topologia o nel costo di un collegamento
Seppur rispondono meglio ai cambiamenti della rete, sono anche maggiormente soggetti ai problemi quali:
 - l'instradamento in loop e
 - l'oscillazione dei percorsi
- Se il criterio è la **sensibilità in base al carico della rete**, allora abbiamo:
 - **Algoritmi sensibili al carico** (load-sensitive)
Che associano i costi (che variano dinamicamente) dei collegamenti al livello corrente di congestione, evitando quelli con un alto costo
 - **Algoritmi insensibili al carico** (load-insensitive)
Che NON associano i costi dei collegamenti al livello corrente di congestione, dato che il costo di un collegamento non riflette esplicitamente il suo attuale o recente livello di congestione

5.2.1 - Instradamento "link-state" (LS)

Ripasso

Algoritmi centralizzati

Calcolano i percorsi a costo minimo tra una sorgente e una destinazione avendo una conoscenza globale e completa della rete, che si traduce nell'avere informazioni complete su connettività (quali router hanno vicino) e costi (qual è il costo per collegarsi ai router adiacenti), tanto da farli valere il nome di **algoritmi link-state (LS)**

Per aver nota la tipologia della rete e tutti i costi dei collegamenti vengono svolti due passaggi:

1. Costruzione della routing table

Vengono inviati a ciascun nodo dei pacchetti in maniera iterativa sullo stato dei suoi collegamenti a tutti gli altri nodi della rete, pacchetti che contengono identità e costi dei collegamenti connessi al nodo che li invia (chi è collegato a chi e quanto costa connettersi a uno)

Quanto descritto viene fatto attraverso un algoritmo di **link-state broadcast** (Ricorda questo nome, **protocollo OSPF**, 5.3), così che tutti i router dispongano di una visione identica e completa della rete e ciascun nodo, che esegue l'algoritmo LS, ottenga gli stessi risultati, il tutto grazie a una tabella, nonché la routing table, che riepiloga le identità e costi scambiati

I messaggi che vengono mandati non derivano da un solo router, bensì sono intercambiati tra i nodi della rete, una propagazione che prende il nome di “flooding”

NOTA - Impossibilità di ricevere lo stesso messaggio

Si potrebbe pensare che nel propagare questi messaggi un nodo possa ricevere lo stesso più volte, tuttavia, ogni router ha un contatore dei messaggi già inoltrati, così che, se gliene arriva uno identico, questo viene scartato al fine di non protrarre all'infinito il flooding

2. Calcolo del percorso migliore

Per calcolare il percorso migliore viene fatto uso di un algoritmo di instradamento link-state noto come **algoritmo di Dijkstra**

L'algoritmo in questione è anche esso iterativo e calcola il percorso a costo minimo da un nodo a tutti gli altri della rete dopo n-iterazioni

Algoritmo Dijkstra dal nodo origine u

```
1 Inizializzazione:
2 N' = {u}
3 per tutti i nodi v
4   se v è adiacente a u
5     allora D(v) = c(u,v)
6   altrimenti D(v) = ∞
7
8 Ciclo
9   determina un w non in N' tale che D(w) sia minimo
10  aggiungi w a N'
11  aggiorna D(v) per ciascun nodo v adiacente a w e non in N':
12    D(v) = min (D(v), D(w) + c(w,v))
13  /* il nuovo costo verso v è il vecchio costo verso v
14     oppure
15     il costo del percorso minimo noto verso w più il
16     costo da w a v */
17  ripeti in ciclo finché non si verifica che N' = N
```

Per comprendere al meglio il suo funzionamento, anche in riferimento all'immagine, definiamo la seguente notazione:

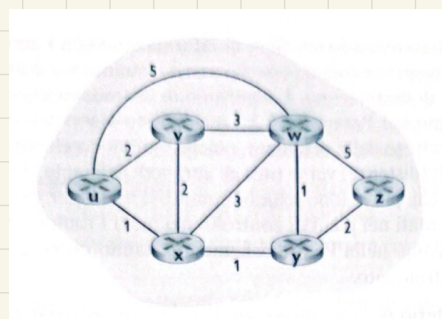
Ripasso

Per ogni arco (x, y) tra i nodi x e y denotiamo con $c(x, y)$ il suo costo, costo che:

- se la coppia (x, y) appartiene ad E, $c(x, y)$ = un valore
- se la coppia (x, y) NON appartiene ad E, $c(x, y)$ = ∞

- $D(v)$: stima del costo minimo del percorso dall'host origine a quello destinatario
- $p(v)$: nodo precedente lungo il percorso dall'host origine a quello destinatario
- N' : sottoinsieme di nodi di cui si conosce il costo minimo nel percorso

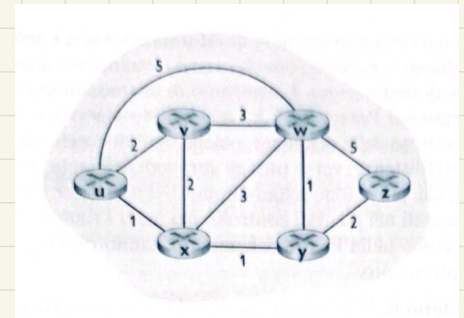
Passo	N'	D(v), p(v)	D(w), p(w)	D(x), p(x)	D(y), p(y)	D(z), p(z)
0	u	2, u	5, u	1, u	∞	∞
1	ux	2, u	4, x		2, x	∞
2	uxy	2, u	3, y			4, y
3	uxyv		3, y			4, y
4	uxyvw					4, y
5	uxyvwz					



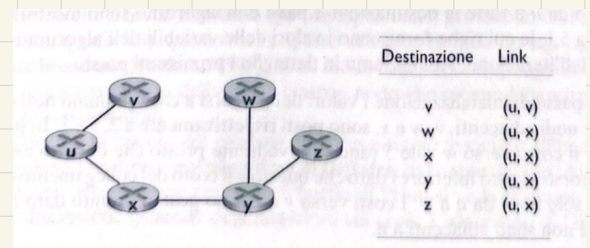
A questo punto, possiamo dire che il calcolo del percorso avviene in tre passaggi ripetuti ciclicamente:

1. Partendo da un router (u), si inserisce in questo in N' per poi raccogliere tutti i costi dei collegamenti tra questo e gli altri nodi della rete
2. Presi tutti i nodi della rete, si seleziona il nodo con il più basso costo di collegamento (x) nella sezione N' insieme al nodo di partenza
3. Dal router da cui abbiamo preso il costo più piccolo (x), riprendiamo i costi dei collegamenti, ma stavolta dei router vicini a lui (eccetto per quelli già presenti in N')

Passo	N'	D(v), p(v)	D(w), p(w)	D(x), p(x)	D(y), p(y)	D(z), p(z)
0	u	2, u	5, u	1, u	∞	∞
1	ux	2, u	4, x		2, x	∞
2	uxy	2, u	3, y			4, y
3	uxyv		3, y			4, y
4	uxyvw					4, y
5	uxyvwz					



Ripetendo questi passaggi n-volte si otterrà il percorso migliore tra un router e l'altro o il cosiddetto "albero dei percorsi minimi)



Per quanto riguarda la complessità dell'algoritmo Dijkstra, questo ha complessità computazionale uguale a $n(n+1)/2$, a ragion del fatto che, partendo dal dover elaborare n nodi, man mano che si procede se ne dovranno elaborare n-x, con x l'interazione corrente in cui ci si trova

5.2.2 - Instradamento "distance-vector" (DV)

Ripasso

Algoritmi decentralizzati

Calcolano i percorsi a costo minimo tra una sorgente e una destinazione in modo distribuito e iterativo

Nessun nodo possiede informazioni complete sul costo di tutti i collegamenti della rete:

- inizialmente i nodi conoscono soltanto i costi dei collegamenti a loro incidenti
- poi, attraverso un processo iterativo e lo scambio di informazioni con i nodi adiacenti, un nodo gradualmente calcola il percorso a costo minimo verso una destinazione o un insieme di destinazioni

(Ricorda questo nome, **algoritmo distance-vector (DV)**)

L'algoritmo distance-vector è:

- distribuito
ciascun nodo riceve parte dell'informazione da uno dei suoi vicini direttamente connessi a cui, dopo aver effettuato il calcolo, restituisce i risultati

- iterativo
questo processo si ripete fino a quando non avviene ulteriore scambio informativo tra vicini
- asincrono
non richiede che tutti i nodi operino al passo con gli altri
- auto-terminante
non vi è alcun segnale che il calcolo debba fermarsi, semplicemente si blocca
- basato sull'equazione di Bellman-Ford
dato $d_x(y)$ il costo minimo del percorso dal nodo x al nodo y , allora

$$d_x(y) = \min_v \{c(x, y) + d_v(y)\} \quad \text{con:}$$
 - v indica il vicino al router
 - $\min_v$ è il minimo della somma che riguarda tutti i vicini
 - $c(x, y)$ è il costo da x a y
 - $d_v(y)$ è il costo minimo del percorso da v a y

Algoritmo Bellman-Ford
A ciascun nodo x :

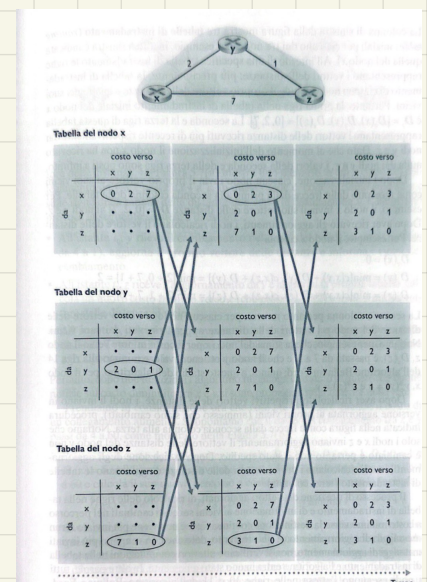
```

1  Inizializzazione:
2  per tutte le destinazioni  $y$  in  $N$ :
3     $D_x(y) = c(x, y)$  /* se  $y$  non è adiacente, allora  $c(x, y) = \infty$  */
4  per ciascun vicino  $w$ 
5     $D_x(y) = ?$  per tutte le destinazioni  $y$  in  $N$ 
6  per ciascun vicino  $w$ 
7    invia il vettore delle distanze  $D_x = [D_x(y): y \text{ in } N]$  a  $w$ 
8
9  ciclo
10  attendi (finché vedi cambiare il costo di un collegamento verso
11    qualche vicino  $w$  o finché ricevi un vettore delle distanze
12    da qualche vicino  $w$ )
13  per ogni  $y$  in  $N$ :
14     $D_x(y) = \min_v \{c(x, v) + D_v(y)\}$ 
15  se  $D_x(y)$  è cambiato per qualche destinazione  $y$ 
16    invia il vettore delle distanze  $D_x = [D_x(y): y \text{ in } N]$  a tutti i vicini
17
18  ripeti il ciclo indefinitamente

```

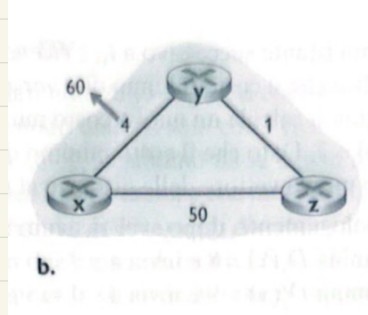
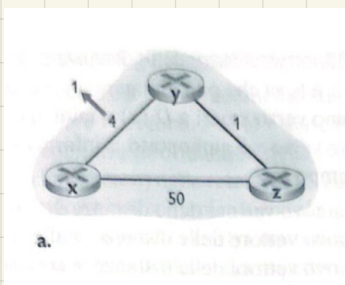
Tale formula fornisce le righe della tabella di inoltro nel nodo di x e l'idea di base è la seguente:

- Ogni nodo crea il suo distance vector, che è impostato come una tabella che riepiloga tutti i costi da affrontare per collegarsi con ognuno dei router della rete
- Una copia di essa viene inoltrata ai router vicini così da essere informati su quanto costa andare dai loro vicini tramite il mittente
- Se la copia ricevuta:
 - è diversa dal loro distance-vector, la tabella viene sistemata, cambiando le stime per poi mandare quella nuova agli altri router
 - è uguale, il router aspetta l'arrivo di altri distance vector



Tuttavia, quando un nodo che esegue l'instradamento DV rileva un cambiamento nel costo dei collegamenti con un vicino, aggiorna il proprio vettore delle distanze e, se si verifica un cambiamento nel costo del percorso a costo minimo, trasmette ai suoi vicini il proprio nuovo vettore delle distanze

Questa situazione può sfociare in due scenari differenti:



- il costo diminuisce, y aggiorna il proprio vettore DV, informa i vicini del cambiamento, tra cui z che ricevuto aggiorna la sua tabella, calcolando un nuovo costo da lui a x di 2 invece che 5, manda aggiornamenti, y riceve aggiorna

Il costo aumenta e a causa della legge di Bellman-Ford che ha dei valori non aggiornati (pensa che da z a x costi 5, ma non sa che era così prima che cambiasse il costo perché non sapeva che passasse per y) y continuerà ad aggiornare la sua tabella con valori errati e via via crescenti finché non si accorgerà dell'errore (**problema del conteggio all'infinito**)

Per risolvere il problema del conteggio all'infinito si fa uso della tecnica **inversione avvelenata** (poisoned reverse), se un router è consapevole che per veicolare una informazione ad un altro router deve passare attraverso un altro, allora informa quest'ultimo che la sua distanza dal destinatario è infinita

- Prima che il costo del collegamento cambi, $D_y(x) = 4$, $D_z(y) = 1$, $D_z(x) = 5$. All'istante t_0 , y rileva che il costo del collegamento è passato da 4 a 60 e calcola il suo nuovo percorso a costo minimo verso x con la formula

$$D_y(x) = \min \{ c(y,x) + D_x(x), c(y,z) + D_z(x) \} = \min \{ 60 + 0, 1 + 5 \} = 6$$

Ovviamente, con la nostra visione globale della rete, possiamo rilevare che questo nuovo costo attraverso z è *errato*. Ma l'unica informazione che il nodo y possiede è che il costo diretto verso x è 60 e che z ha ultimamente detto a y di essere in grado di giungere a x con un costo di 5. Pertanto al fine di arrivare a x, y ora farebbe passare il percorso per z, aspettandosi che questo sia in grado di giungere a x con un costo pari a 5. All'istante t_1 , abbiamo un **instradamento ciclico**: al fine di giungere a x, y fa passare il percorso per z e z lo fa passare per y. Un ciclo in un instradamento assomiglia a un buco nero: un pacchetto destinato a x che arriva a y o z all'istante t_1 rimbalzerà avanti e indietro tra questi due nodi per sempre (a meno che non vengano cambiate le tabelle di inoltro).

- Dato che il nodo y ha calcolato un nuovo costo minimo verso x, informa z del suo nuovo vettore delle distanze all'istante t_1 .

- In un istante successivo a t_1 , z riceve il nuovo vettore delle distanze di y, che indica che il costo minimo di y verso x è 6, sa che può giungere a y a costo 1 e quindi calcola un nuovo costo minimo verso x pari a $D_z(x) = \min \{ 50 + 0, 1 + 6 \} = 7$. Dato che il costo minimo di z verso x è aumentato, z informa y del suo nuovo vettore delle distanze al tempo t_2 .

- Analogamente, dopo aver ricevuto il nuovo vettore delle distanze di z, y determina $D_y(x) = 8$ e invia a z il suo nuovo vettore delle distanze. z allora determina $D_z(x) = 9$ e invia a y il suo nuovo vettore delle distanze, e così via.

Il ciclo proseguirà per 44 iterazioni (scambi di messaggi tra y e z), fino a quando z considera il costo del proprio percorso attraverso y maggiore di 50. A questo punto, z determina che il percorso a costo minimo verso x passa attraverso la connessione diretta a x, e y instraderà verso x passando per z. Il risultato delle cattive notizie sull'incremento del costo dei collegamenti ha viaggiato senza dubbio assai lentamente. Che cosa sarebbe successo se il costo del collegamento $c(y,x)$ fosse passato da 4 a 10.000 e il costo $c(z,x)$ avesse avuto valore 9999? A causa di questi scenari, il processo che abbiamo descritto viene talvolta detto **problema di conteggio all'infinito** (count-to-infinity problem).

Per quanto riguarda la complessità dell'algoritmo DV, non è possibile fare una stima perché dipende dal grafo

A conti fatti i due algoritmi visti utilizzano approcci complementari nel calcolare l'instradamento

- nell'algoritmo LS, ciascun nodo dialoga con tutti gli altri nodi via broadcast, ma comunica loro solo i costi dei collegamenti direttamente connessi
- nell'algoritmo DV, ciascun nodo dialoga solo con i vicini direttamente connessi, informandoli della stima costo minore da se stesso a tutti i nodi che conosce nella rete

Per questa ragione risulta complicato confrontarli, tanto è vero che dal punto di vista della:

- complessità dei messaggi
 - LS richiede che ciascun nodo conosca il costo di ogni collegamento della rete e ogni qualvolta che cambia il costo di un collegamento, il nuovo costo deve essere comunicato a tutti i nodi
 - DV richiede scambi di messaggi tra nodi decenti a ogni iterazione, un'operazione che però, come già detto, non è possibile farne una stirpo in termini di tempo perché dipende dalla complessità del grafo
- velocità di convergenza
 - LS
 - DV può converge lentamente e può presentare cicli instradamento e il problema del conteggio all'infinito
- robustezza
 - con LS, un router può comunicare via broadcast un costo sbagliato, per uno dei suoi collegamenti connessi, ma non per altri, allo stesso tempo il nodo può anche alterare o eliminare i pacchetti ricevuti in broadcast LS, ma i nodi LS si occupano di calcolare soltanto le proprie tabella di inoltro e gli altri nodi effettuano calcoli simili per quanto riguarda, così facendo i calcoli di instradamento sono in qualche modo isolati, il che fornisce un certo grado di robustezza
 - con DV, un nodo può comunicare percorsi a costo minimo in route a tutte le destinazioni

In conclusione nessuno dei due algoritmi è nettamente superiore all'altro ed entrambi vengono utilizzati in Internet

5.3 - Instradamento interno ai sistemi autonomi: OSPF

Nel nostro studio degli algoritmi di routing abbiamo visto la rete in una maniera molto idealizzata perché:

- abbiamo studiato grafi in cui router erano indistinguibili, identici
- abbiamo trattato situazioni che rimanevano immutate nel tempo, senza che si presentasse alcun tipo di problema

Questa trattazione semplificata della rete risulta problematica per una corretta comprensione di come stanno veramente le cose per due motivi:

- scalabilità

Al crescere del numero di router, il tempo richiesto per calcolare, memorizzare e comunicare le informazioni instradamento diventa proibitivo

- Per LS diventerebbe difficoltoso
- Per DV, con un così grande numero di iterazioni da fare tra i router, non convergerebbe mai

- autonomia amministrativa

Da un punto di vista ideale, ciascuno dovrebbe essere in grado di amministrare la propria rete nel modo auspicato, pur mantenendo la possibilità di connetterla alle reti esterne

Per risolvere questi problemi si organizzano i router in **sistemi autonomi (AS, Autonomous System)**, generalmente, gruppi di router posti sotto lo stesso controllo amministrativo (tipicamente L'ISP), sotto cui eseguono lo stesso algoritmo di instradamento e gli uni hanno informazioni sugli altri

Parlando di algoritmi di instradamento:

- L'algoritmo di instradamento in esecuzione in un AS è detto **protocollo di instradamento interno al sistema autonomo (intra-AS routing protocol)**

Essendo eseguito nello stesso AS ogni amministratore può decidere un proprio algoritmo di routing

- L'algoritmo di instradamento in esecuzione su AS diversi è detto **protocollo di instradamento tra sistemi autonomi (inter-AS routing protocol)**

In merito a questo ultimo punto:

- finché gli AS sono connessi via un solo collegamento tramite **router gateway**, i protocolli di instradamento tra essi non devono per forza essere gli stessi
- se gli AS sono connessi via più collegamenti tramite **router gateway**, in quel caso è necessario l'utilizzo di un algoritmo di instradamento comune

Alla fine dei conti, indipendentemente dagli algoritmi utilizzati, la Forwarding table è frutto di una cooperazione tra intra-AS e Inter-AS routing protocol

Ritornando a parlare di Inter-AS routing, di seguito una lista dei protocolli usati:

- RIP, Routing Information Protocol

Uno dei primi protocolli Internet

Di tipo DV il quale scambia i distance-vectors ogni 30 secondi

Non più ampiamente usato rispetto al passato

- EIGRP, Enhanced Interior Gateway Routing Protocol

Di tipo DV

Per decenni proprietà di CISCO

- OSPF, Open Shortest Path First

Di tipo LS

Usa il flooding per i messaggi di stato e l'algoritmo Dijkstra per determinare il percorso a costo minimo

Rinomato per:

- sicurezza

Gli scambi tra router OSPF possono essere autenticati, criptati, di conseguenza soltanto router fidati possono prendere parte al protocollo OSPF in un AS evitando che malintenzionati mettano informazione errata nelle tabelle dei router

- percorsi con lo stesso costo

Quando i percorsi verso la destinazione hanno lo stesso costo, OSPF consente di usarli senza dovere sceglierne uno per trasportare tutto il traffico

- Supporto integrato per l'instradamento unicast o multicast

- una connessione TCP è anche detta **punto a punta** (point-to-point), ossia ha luogo tra un singolo mittente e un singolo destinatario (Multicast, trasferimento dati da un singolo mittente a più destinatari non possibile con TCP)

- Supporto alle gerarchie di un dominio di instradamento

Possibilità di strutturare gli AS in modo gerarchico

Per far sì in modo che un sistema autonomo funzioni al meglio delle sue capacità, lo si divide in ancora più parti, dette aree, le quali a loro volta sono collegate a un'altra che prende il nome di backbone

Ogni area avrà la sua topologia e il suo algoritmo di routing locale, ma per mandare qualcosa alle altre dovrà rifarsi alla backbone

5.4 - Instradamento tra ISP: BGP

Ripasso

- L'algoritmo di instradamento in esecuzione in un AS è detto **protocollo di instradamento interno al sistema autonomo (intra-AS routing protocol)**
- L'algoritmo di instradamento in esecuzione su AS diversi è detto **protocollo di instradamento tra sistemi autonomi (inter-AS routing protocol)**

OSPF

BGP, Border Gateway Protocol

Il collante che tiene insieme le migliaia di ISP che formano Internet

Gira su TCP e permette alla subnet di propagare le sue informazioni al resto di internet, del fatto che esiste e chi può raggiungere

“Sono qui, da qui io posso raggiungere chi e come”

5.4.1 - Il ruolo di BGP

BGP mette a disposizione di ciascun router un modo per:

1. Ottenere informazioni sulla raggiungibilità delle subnet da parte degli AS confinanti

L'AS comunicando a BGP la propria esistenza, quest'ultimo si occuperà di farla sapere agli altri AS confinanti

2. Determinare i percorsi ottimi verso le sottoreti

Un router può venire a conoscenza di più cammini verso uno specifico prefisso e per terminare il migliore segue localmente BGP sulla base delle informazioni di raggiungibilità e delle politiche del sistema

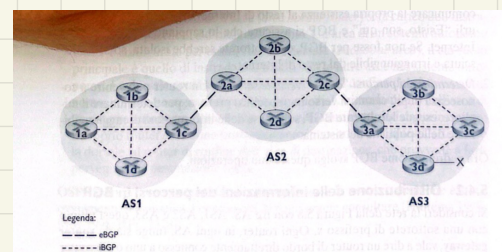
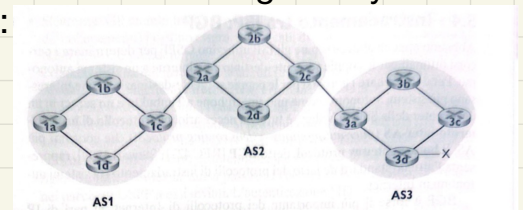
5.4.2 - Distribuzione delle informazioni dei percorsi in BGP

BGP provvede ad ogni sistema autonomo e si divide in:

- eBGP, esterna, quella parte di protocollo che permette ai router di bordo di un sistema autonomo di parlare con altri router di bordo di sistemi autonomi
- iBGP, interna, quella parte di protocollo che permette ai router di bordo di un sistema autonomo di parlare con altri router interni di sistemi autonomi

Le sessioni BGP sono tutte le connessioni TCP con tutti i messaggi BGP che vengono inviati

Router gateway = router BGP



BGP consente di conoscere quali sono le destinazioni raggiungibile attraverso AS vicini

In BGP le distanze non sono host, ma piuttosto prefissi CIDR (ID di rete e net-mask) che rappresentano una sottorete o una collezione di sottoreti

5.4.3 - Selezione delle rotte migliori

Quando un router annuncia un prefisso per una sessione BGP, include anche un certo numero di **attributi BGP**, che in gergo, un prefisso assieme ai suoi attributi è detto **rotta** (route)

Due dei più importanti attributi sono:

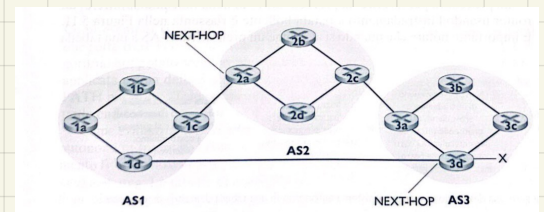
- **AS-PATH**

Questo attributo elenca gli AS attraverso i quali è passato l'annuncio del prefisso

Quando un prefisso attraverso un AS, questi aggiunge il proprio AS number all'attributo AS-PATH

- **NEXT-HOP**

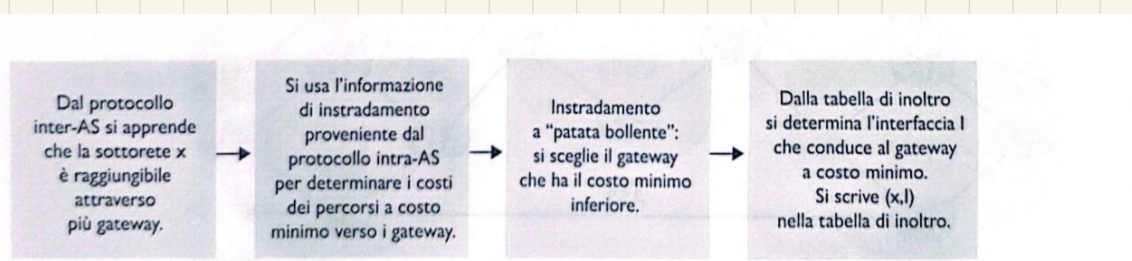
È l'interfaccia del router che inizia l'AS-PATH e tale attributo viene usato per configurare correttamente le loro tabelle di inoltramento



Usando i valori di NEXT-HOP e l'algoritmo di instradamento intra-AS, il router può determinare il costo del percorso verso due collegamenti applicare l'instradamento per determinare l'interfaccia appropriata

Instradamento a patata bollente (hot-potato routing)

Il AS si sbarazza del pacchetto (la patata bollente) non appena possibile, visione del nodo meno costoso possibile, ciò avviene facendo inviare il pacchetto da un router gateway con il minor costo



Quando un router gateway riceve un annuncio di rotta, utilizza le proprie **politiche di importazione**, per decidere se accettare o filtrare la rotta e se impostare determinati attributi quali i parametri di scelta. Le politiche di importazione possono cancellare una rotta, perché il sistema autonomo potrebbe non voler inviare traffico su uno dei sistemi autonomi nell'AS-PATH del percorso, oppure un gateway può ignorare una rotta perché conosce una via migliore verso lo stesso prefisso.

Se esistono due o più percorsi verso lo stesso prefisso, BGP invoca in sequenza le seguenti regole di eliminazione fino a individuare un'unica possibilità.

- Alle rotte viene assegnato come attributo un valore di preferenza locale, che potrebbe esser stato impostato direttamente dal router o appreso da un altro router nello stesso AS: Si tratta di una scelta che è lasciata all'amministratore di rete del sistema autonomo, Si selezionano quindi le rotte con i più alti valore di preferenza locale.
- Tra le rotte con lo stesso valore di preferenza locale, si seleziona quella con AS-PATH più breve. Se questa fosse l'unica regola di selezione, allora BGP utilizzerebbe un algoritmo DV per la determinazione del percorso, con metrica sulla distanza data dal numero di hop tra sistemi autonomi anziché il numero di hop tra router
- Tra le rotte con lo stesso valore di preferenza locale e la stessa lunghezza AS-PATH si seleziona quella il cui router di NEXT-HOP è più vicino, cioè che presenta il percorso con costo minore, determinato dall'algoritmo intra-AS.